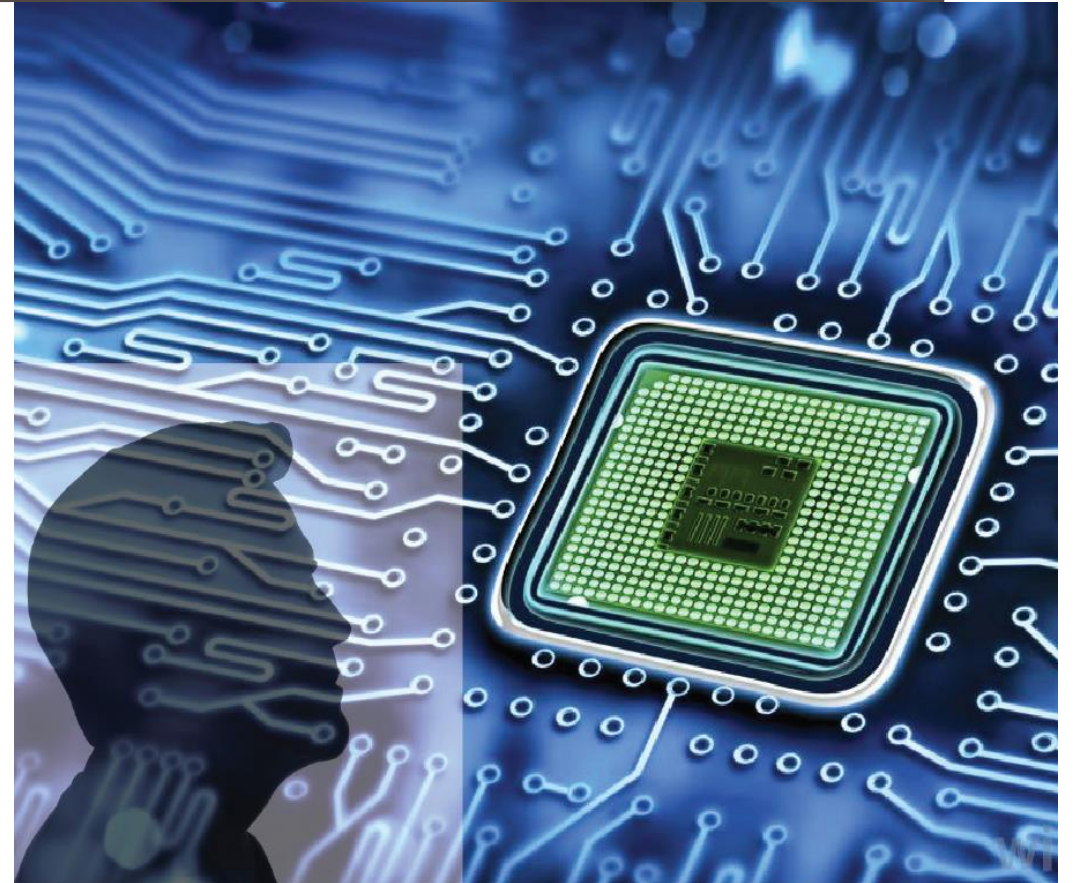


VLSI DESIGN FLOW

Week-7 Lecture-1 Formal Verification

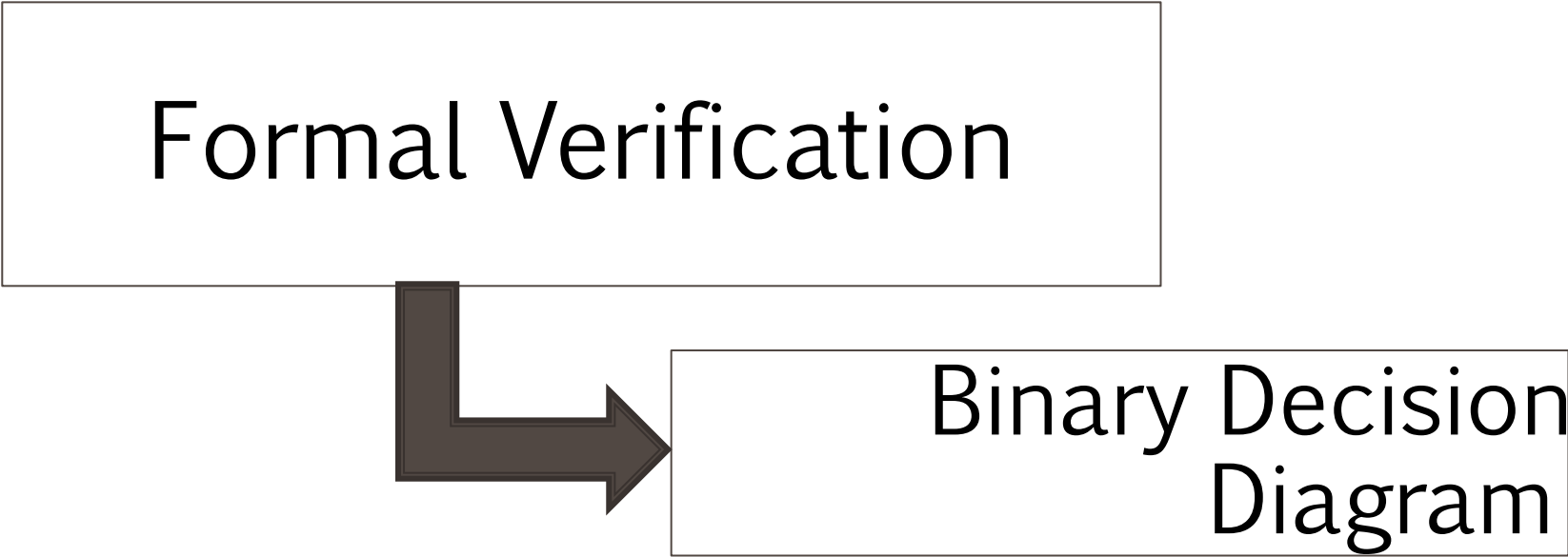
Sneh Saurabh
23rd September, 2024



VDF: Revision/Announcements

-





```
graph TD; A[Formal Verification] --> B[Binary Decision Diagram]
```

Formal Verification

Binary Decision
Diagram

Binary Decision Tree

Boolean functions can be represented as **Binary Decision Tree** by applying **Shannon Expansion Theorem** recursively

$$y = x_1x_2 + x_2'x_3 + x_1'x_3'$$

Level 3: (Expand w.r.t x_3)

$$y_{x_1=0,x_2=0,x_3=0} = 1$$

$$y_{x_1=0,x_2=0,x_3=1} = 1$$

$$y_{x_1=0,x_2=1,x_3=0} = 1$$

$$y_{x_1=0,x_2=1,x_3=1} = 0$$

$$y_{x_1=1,x_2=0,x_3=0} = 0$$

$$y_{x_1=1,x_2=0,x_3=1} = 1$$

$$y_{x_1=1,x_2=1,x_3=0} = 1$$

$$y_{x_1=1,x_2=1,x_3=1} = 1$$

Level 1: (Expand w.r.t x_1)

$$y_{x_1=0} = x_2'x_3 + x_3'$$

$$y_{x_1=1} = x_2 + x_2'x_3$$

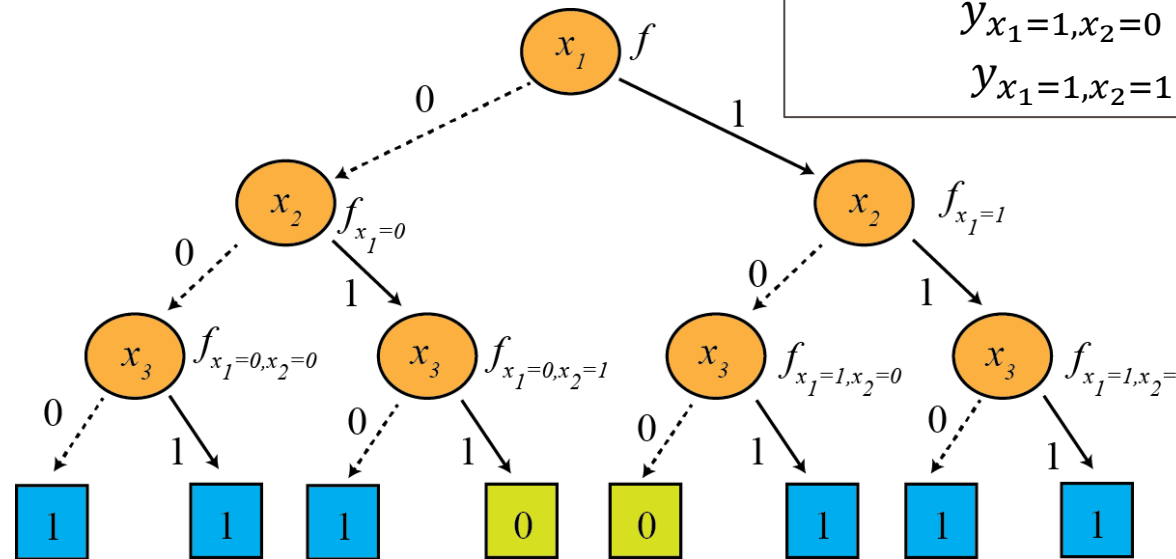
Level 2: (Expand w.r.t x_2)

$$y_{x_1=0,x_2=0} = x_3 + x_3'$$

$$y_{x_1=0,x_2=1} = x_3'$$

$$y_{x_1=1,x_2=0} = x_3$$

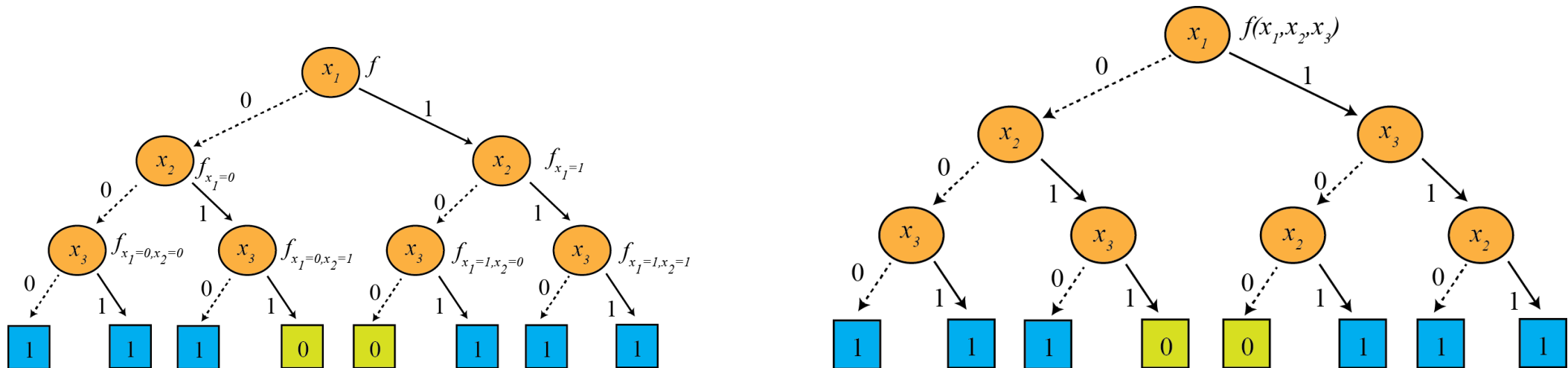
$$y_{x_1=1,x_2=1} = 1$$



To make it canonical and compact, we need to add constraint on variable ordering and remove redundancies

Binary Decision Tree: Variable Order

- At an intermediate node, we can choose any variable for expansion in a binary decision tree
- In general, different variable orders can give different binary decision tree



To enforce canonicity, we need to add constraint on the Binary Decision Tree

Ordered Binary Decision Diagram (OBDD): A BDD becomes an ordered binary decision diagram (OBDD) if the decision variables follow the same order in all the paths from the root to the leaf nodes

Ordered Binary Decision Diagram (OBDD)

- Consider a Boolean function $f(x_1, x_2, x_3, \dots, x_n)$ of n Boolean variables $\{x_1, x_2, x_3, \dots, x_n\}$
- OBDD is a rooted, directed, acyclic graph consisting of two types of vertices:
 - Terminal vertices
 - Non-terminal vertices

Terminal vertices (outdegree is 0):

- 0 Node : function assumes a value “0”
- 1 Node : function assumes a value “1”

Each non-terminal vertex v has:

- Two children $low(v)$ and $high(v)$
- Attribute $index(v) \in \{1, 2, \dots, n\}$ and it refers to the corresponding variable in the set $\{x_1, x_2, x_3, \dots, x_n\}$

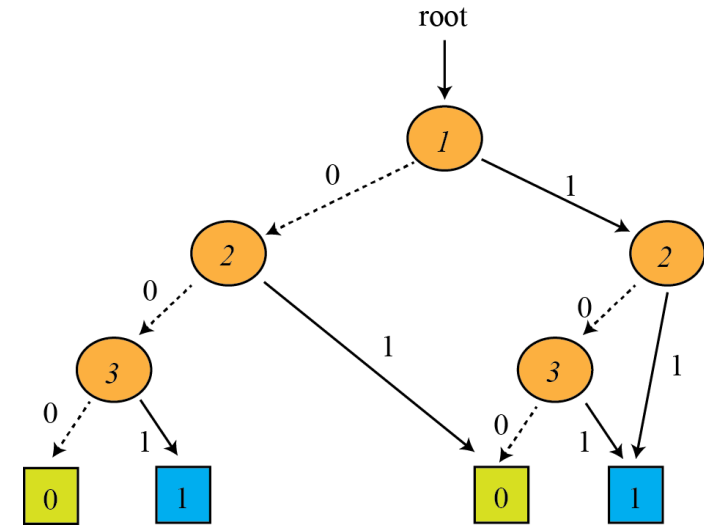
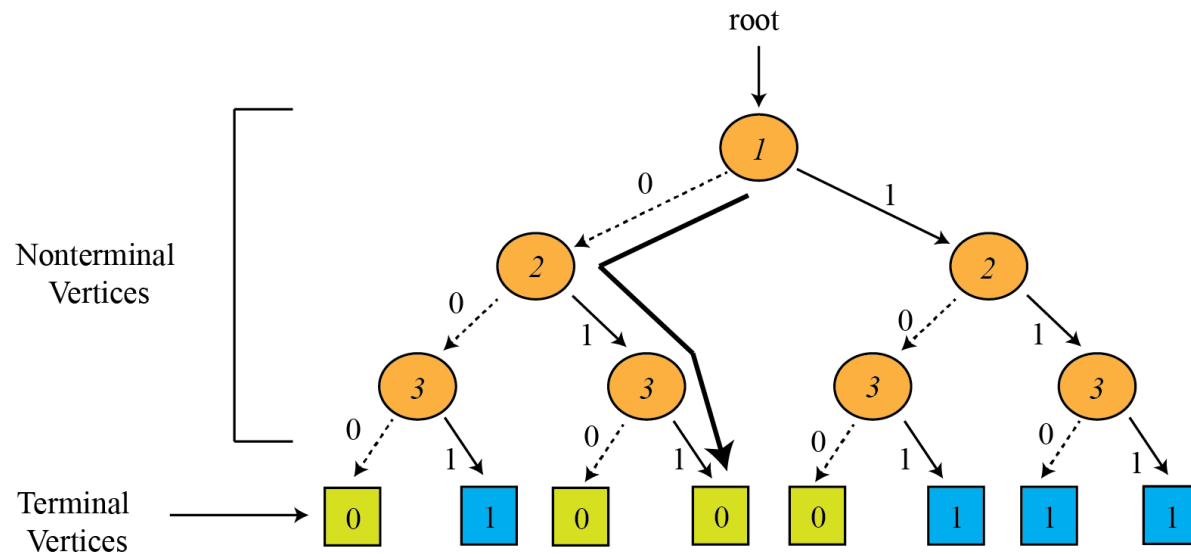
- Edge between a vertex v and $low(v)$ represents the case when the corresponding variable assumes $x_{index(v)} = 0$
- Similarly $high(v)$ corresponds to $x_{index(v)} = 1$

- To enforce ordering of non-terminal vertices: $index(v) < low(index(v))$ and $index(v) < high(index(v))$

OBDD: Example

$$y = x_1x_2 + x_1x_3 + x_2'x_3$$

- A Boolean function does not have a unique OBDD (i.e. OBDD is not canonical)

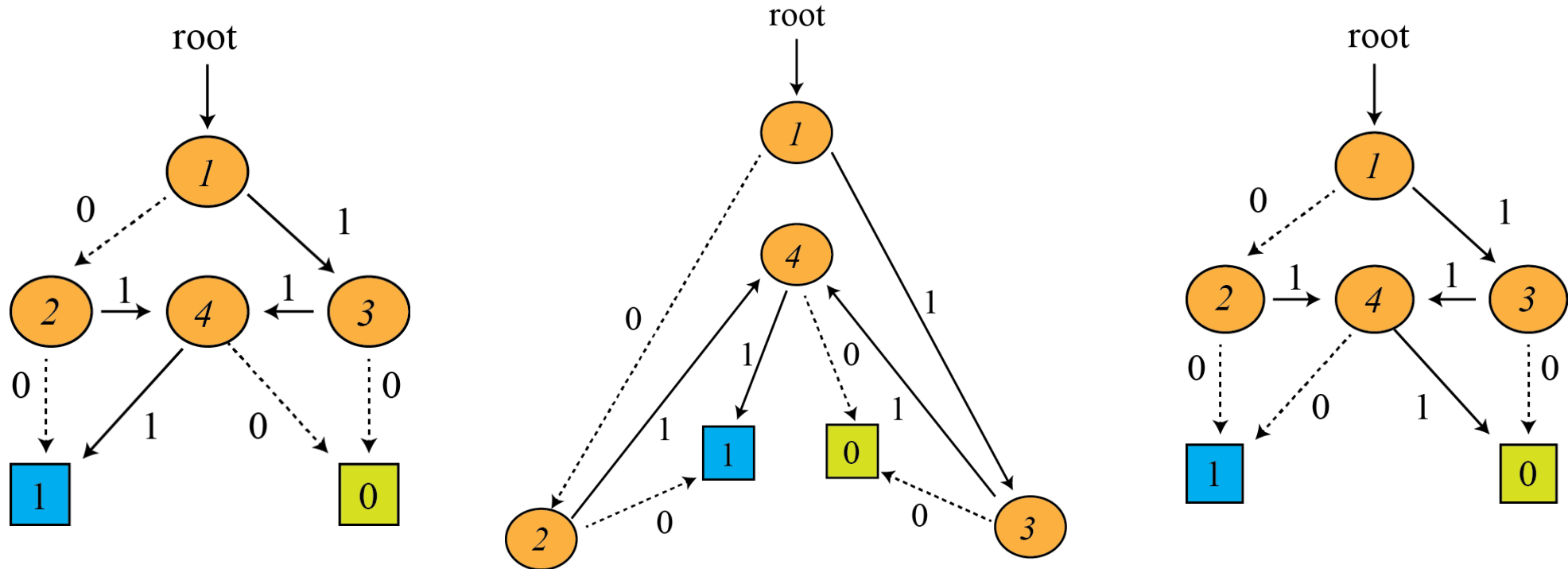


- OBDD can be made canonical by removing redundancies
 - Obtain a Reduced OBDD (ROBDD)

Isomorphic OBDDs

Isomorphism:

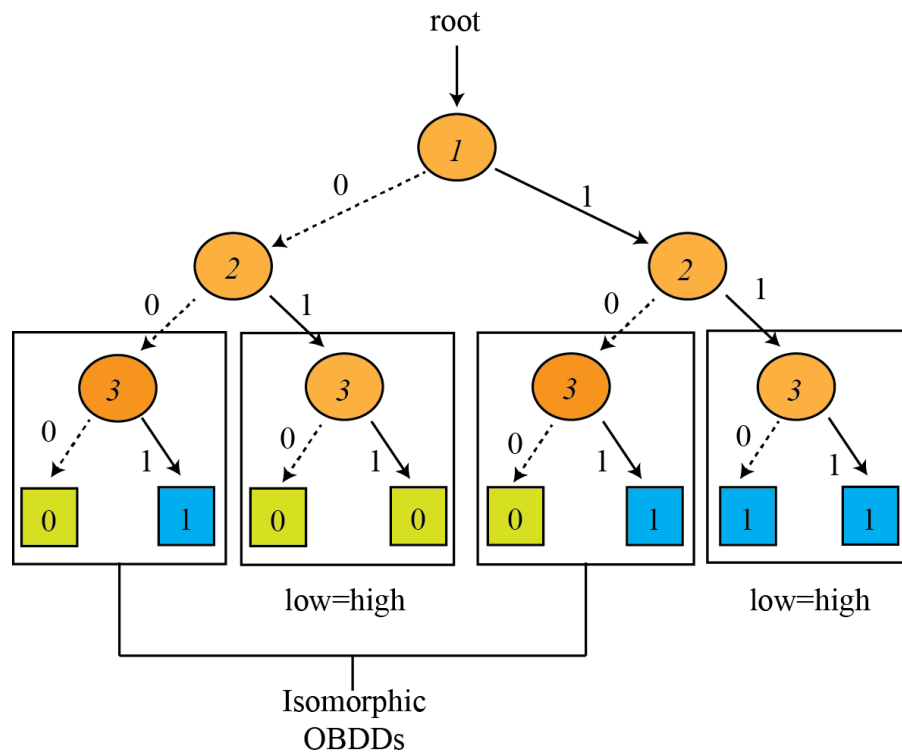
- Two OBDDs F1 and F2 are isomorphic if there exists a one-to-one mapping between their set of vertices such that the adjacency is preserved.
- The correspondence of the value at the terminal vertices and index at the nonterminal vertices must exist.



Reduced Ordered Binary Decision Diagram (ROBDD)

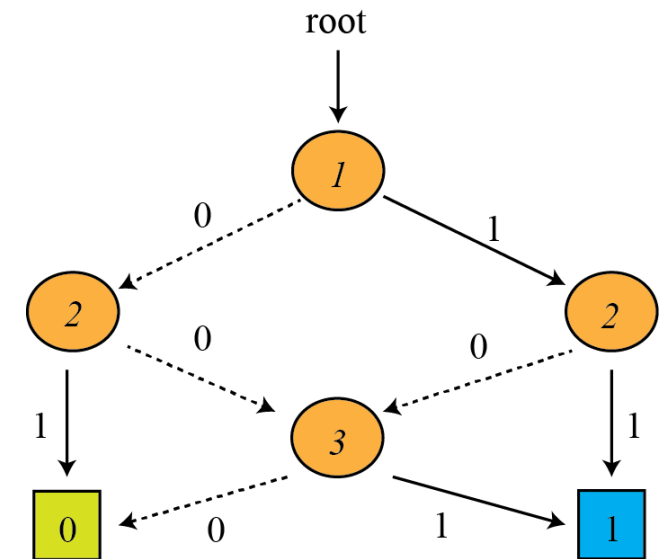
ROBDD is an OBDD with the following constraints:

- No vertex v has $low(v) = high(v)$
- No pair of vertices $\{u, v\}$ exists in the OBDD such that subgraph rooted at u and v are isomorphic.



Can be obtained by systematically removing vertices from the OBDD:

- Any vertex with identical children is removed and replaced with any of its children.
- Two vertices with identical OBDDs are merged into one.



Reduced Ordered Binary Decision Diagram (ROBDD)

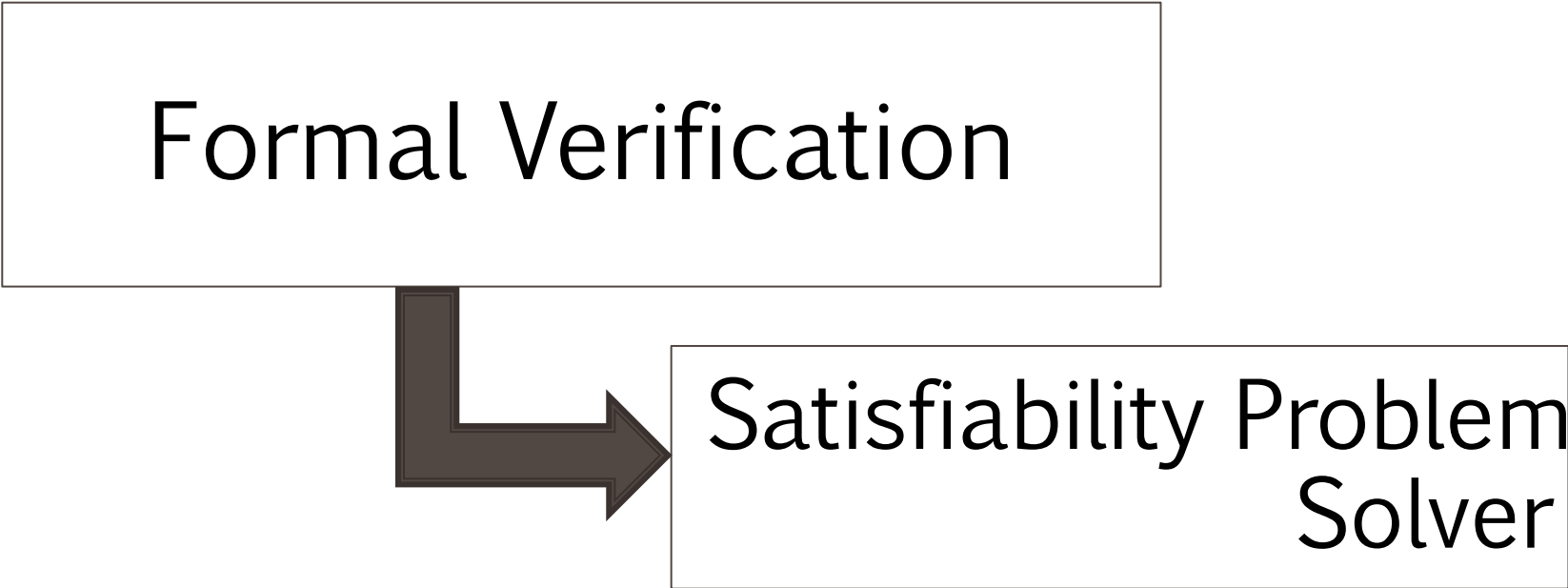
- It can be proven that ROBDD is canonical
- When we talk of BDD, we typically refer to ROBDD

Applications:

- Testing equivalence of two functions is easy
- Testing satisfiability and tautology becomes easy

Compactness:

- Can capture an exponential number of paths from the root to the leaf using a linear number of vertices
 - Size of ROBDDs for many Boolean functions grows as polynomial with the number of variables
- Size of ROBDDs for a given function depends on the variable order.
 - Depending on the variable order, the size of ROBDD can be linear or exponential for an adder circuit
 - Finding a good variable order is difficult
 - For some functions, such as a multiplier, the size of ROBDD is always exponential



```
graph TD; A[Formal Verification] --> B[Satisfiability Problem Solver]
```

Formal Verification

Satisfiability Problem
Solver

Satisfiability : Problem Definition (1)

- $y = f(x_1, x_2, x_3, \dots, x_n)$ where the variables $\{x_1, x_2, x_3, \dots, x_n\}$ are Boolean variables
- Can y be evaluated to 1 by any assignment of variables $\{x_1, x_2, x_3, \dots, x_n\}$?
- If yes, then f is a satisfiable (SAT) instance, else it is an unsatisfiable (UNSAT) instance.
- If f is a SAT instance, one or several assignments that make f evaluate to 1 need to be produced by a SAT solver depending on the application.

Given: $f(x_1, x_2, x_3) = x_1x_2 + x_1x_3 + x_2'x_3$. Is it a SAT instance?

- The function f can be evaluated to 1 by following assignments:
 - $x_1 = 0, x_2 = 0, x_3 = 1$
 - $x_1 = 1, x_2 = 0, x_3 = 1$
 - $x_1 = 1, x_2 = 1, x_3 = 0$
 - $x_1 = 1, x_2 = 1, x_3 = 1$
- Yes, it is a SAT instance

Given: $g(x_1, x_2, x_3) = (x_1 + x_2)(x_1' + x_2')(x_1' + x_2)(x_1 + x_3)(x_1 + x_3')$. Is it a SAT instance?

- The function g cannot be evaluated to 1 for any combination of values of $\{x_1, x_2, x_3\}$ (can be checked by constructing a truth table).
- It is an UNSAT instance

Satisfiability: Problem Formulation

Inputs to SAT solvers are typically given in Conjunctive Normal Form (CNF)

- CNF is AND of clauses
- Clauses are OR of literals
- Literals are variable or its complement

$$f(x_1, x_2, x_3) = (x_1 + x_2)(x_1' + x_2)(x_1 + x_3')$$

- Variables: x_1, x_2, x_3
- Literals: x_1, x_2, x_1', x_3'
- Clauses: $(x_1 + x_2)$, $(x_1' + x_2)$ and $(x_1 + x_3')$

Why to use CNF in SAT solver?

- It reduces to 0 if any of the clauses is 0.
 - To make a function satisfiable, all its clauses must be made 1
- SAT Solvers can exploit this observation
 - Easily detect conflicts
 - Apply reasoning and reduce search space
- A given combinational logic circuit can be transformed into a CNF representation in linear time and space

Satisfiability : k-SAT problem

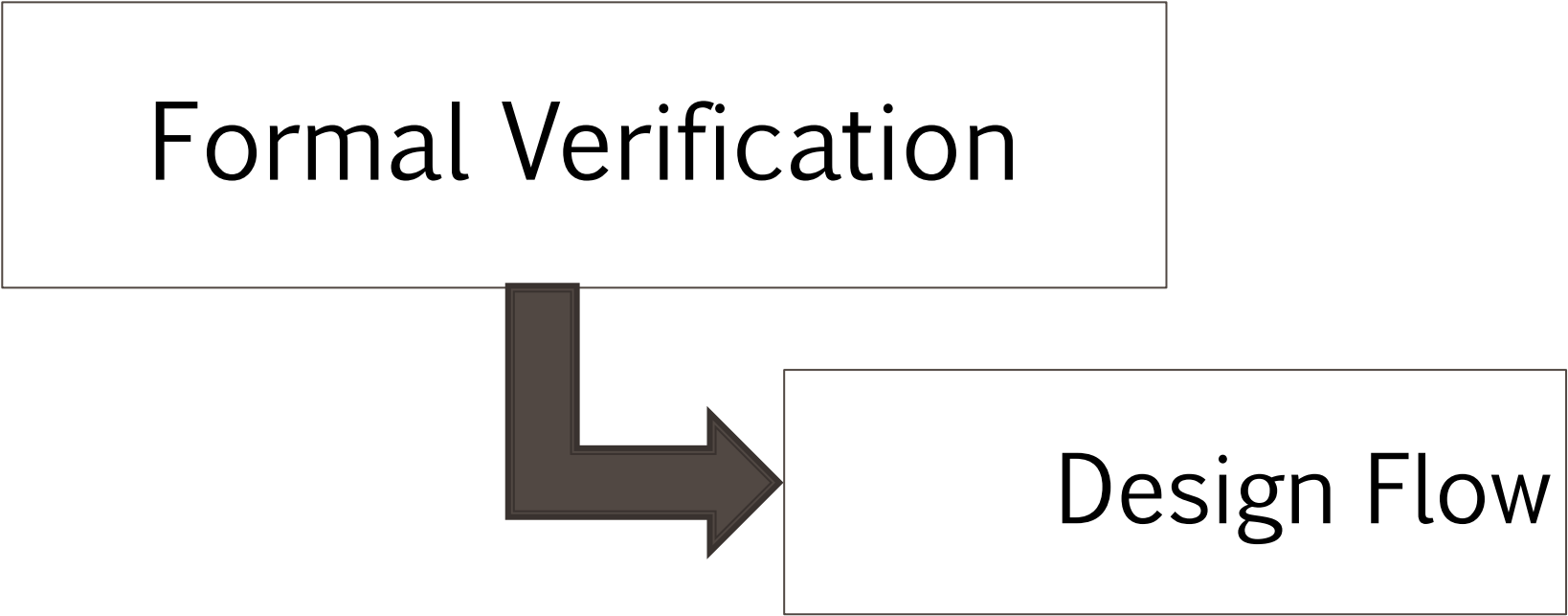
- We encounter various forms of SAT problems
- **k-SAT problem:** each clause in the CNF representation of a Boolean function is of maximum k literals

Examples

- 2-SAT Problem: $f(x_1, x_2, x_3) = (x_1 + x_2)(x_1' + x_2)(x_1 + x_3')$
- 3-SAT Problem: $f(x_1, x_2, x_3) = (x_1 + x_2 + x_3)(x_1' + x_2)(x_1 + x_3')$
- 4-SAT Problem: $f(x_1, x_2, x_3, x_4) = (x_1 + x_2 + x_3)(x_1' + x_2)(x_1 + x_2 + x_3' + x_4')$

Complexity:

- 2-SAT Problem: can be solved in polynomial time
- 3-SAT Problem: NP-complete problem (No known algorithm exist that can solve in polynomial time for the worst case)
- k-SAT where $k > 3$: NP-complete

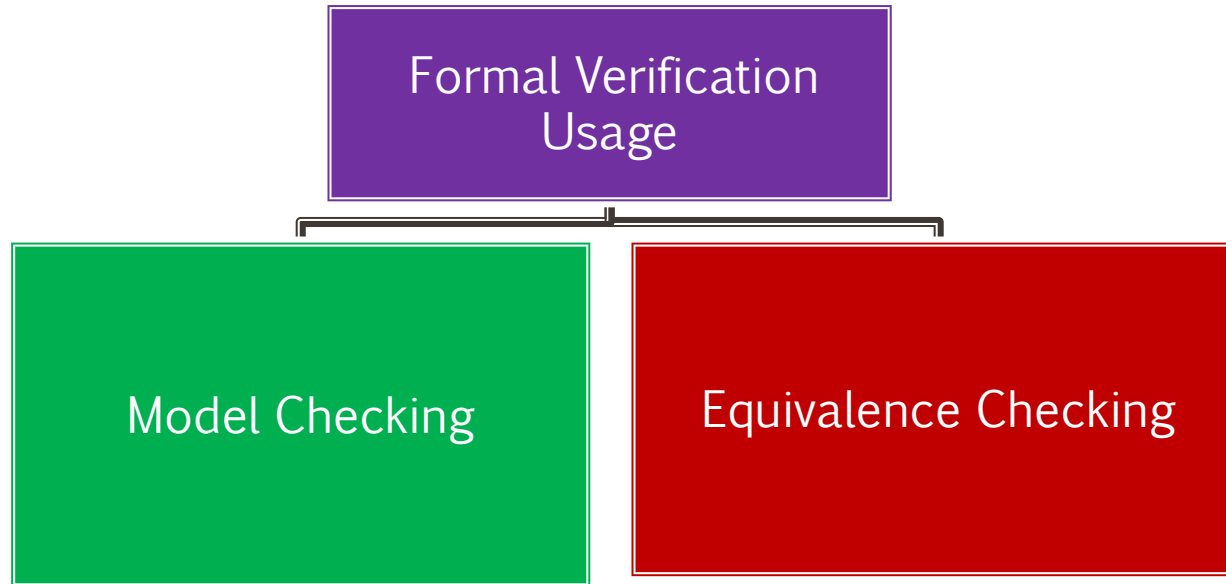


```
graph TD; A[Formal Verification] --> B[Design Flow]
```

Formal Verification

Design Flow

Formal Verification Usage

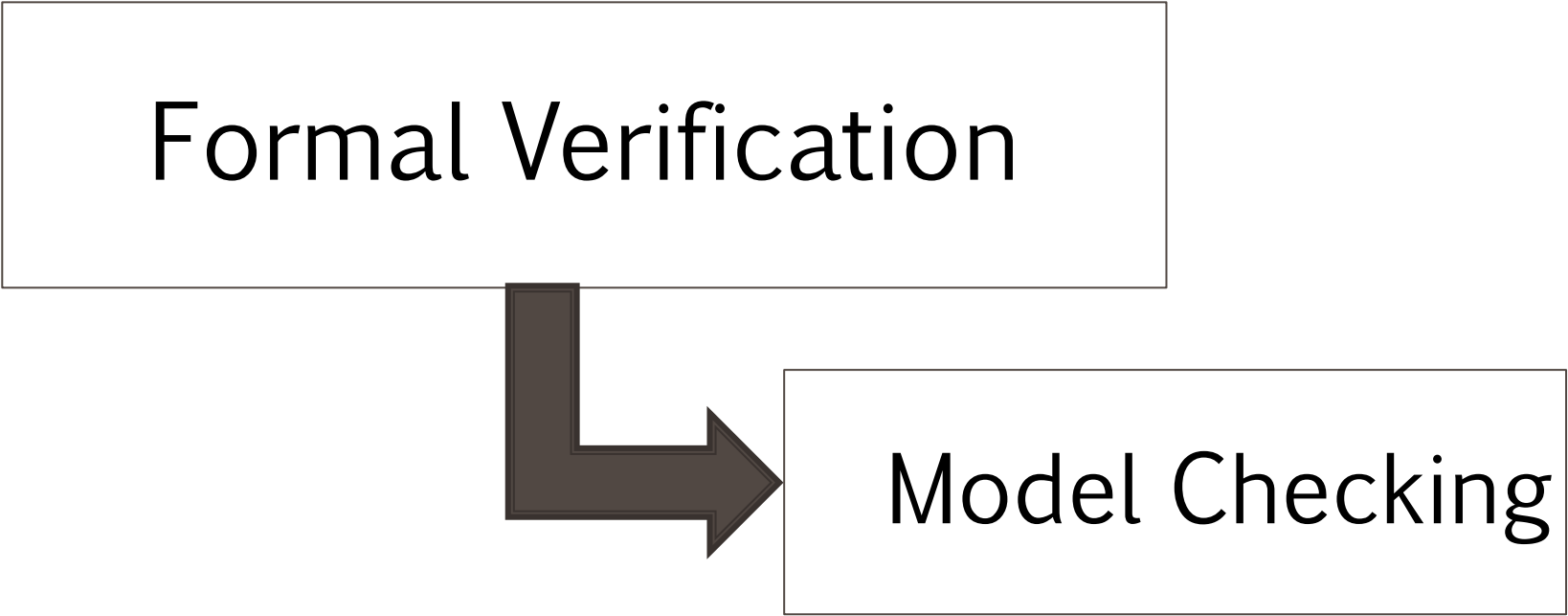


Model Checking

- Verifies that for a given model (design), whether the specifications or given set of properties are satisfied

Equivalence Checking

- Verifies that the two representations of the same design will exhibit exactly the same behavior

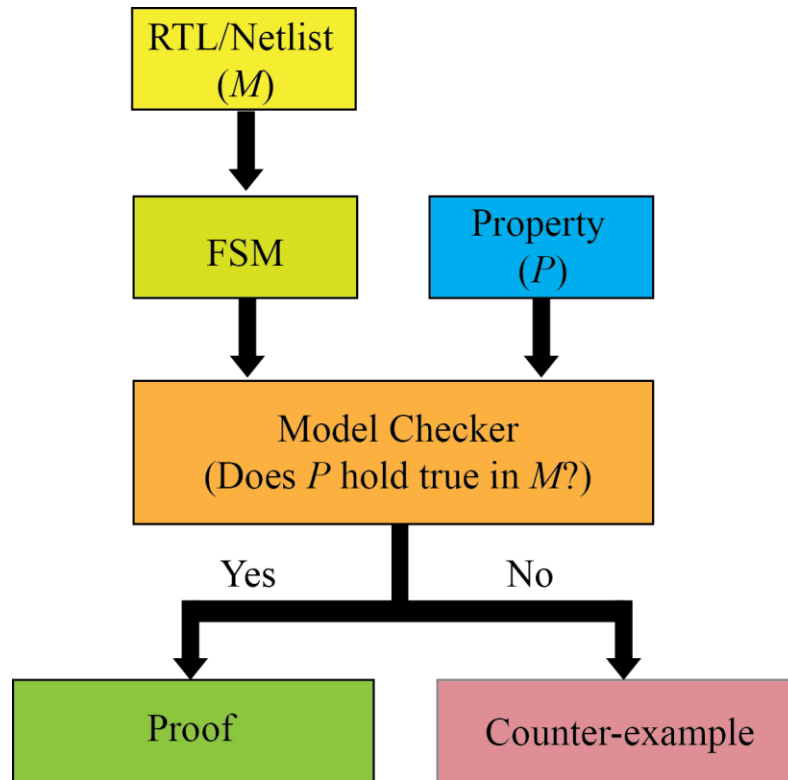


```
graph TD; A[Formal Verification] --> B[Model Checking]
```

Formal Verification

Model Checking

Model Checking: Framework



Inputs

- Given design in RTL or Netlist
 - Typically modelled as an FSM
 - For N state elements: number of states can be 2^N
- Set of properties that a model checker needs to verify
 - Model checker needs to verify that the given property is valid as the system evolves through these states
 - This problem is known as the **state explosion problem**

Model Checking: Property Specification

- Properties are specified in Temporal Logic

Temporal Logic

- System properties need time-related specification, in addition to logic-related expressions

Examples of temporal properties

- *Whenever* a correct password is entered, the door *eventually* opens
- For traffic lights at a given post, one of the red, yellow or green light is *always* ON
- *Whenever* a request is made by multiple requesters, it is *never* granted to more than one requesters simultaneously

- Ordering of events is implicit
 - eventually, never, always, whenever, etc.
- Modelled using System Verilog Assertion (SVA)
 - Embed SVA in RTL
 - Checked by both model checker and simulators
 - Can specify assumptions and constraints (to make property checking easier for the tool)

Model Checking: Techniques

Primary Difficulty:

- Exhaustive search becomes difficult due to the state explosion problem
- Explicitly enumerating states and representing them as graphs could not scale to be practically useful
- A breakthrough came around 1990 when symbolic state-space exploration was proposed :
 - First it employed BDDs
 - Later SAT-based techniques.

References

- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.